

Cryptographic Protection of Machine Learning as a Service

DSC 4110/6111: Modern Cryptography and AI/ML Security

Dr. Laltu Sardar

School of Data Science,
Indian Institute of Science Education and Research Thiruvananthapuram (IISER TVM)



September 30, 2026

Table of Content

1 From Bit Commitment and ZKP to Secure Computation

From Bit Commitment and ZKP to Secure Computation

From Bit Commitment and ZKP to MLaaS

Previous classes:

- Bit commitment
 - Commit now, reveal later
 - Hides the committed value
 - Prevents changing the value later
- Zero-Knowledge Proofs (ZKP)
 - Prove a statement without revealing the secret
 - Useful for proving correctness or knowledge

Question: How can these cryptographic ideas help protect data in ML services?

Machine Learning as a Service (MLaaS)



- **Model Owner:** trains and deploys the ML model
- **Cloud Server:** provides ML inference as a service
- **User:** sends data/query and receives prediction

What Can Go Wrong?

We must first ask:

Who can be trusted?

What happens if one of the participants does not follow the protocol?

- Model may be confidential
- User data may be sensitive
- Query may reveal private information
- Server may learn information from queries
- Server may return an incorrect result

Therefore, we need an adversarial model.

Three Types of Adversaries

Trusted

The party follows the protocol and does not intentionally violate security.

We assume the party behaves correctly.

Semi-Honest

The party follows the protocol correctly, but tries to learn additional information.

Correct execution, curious behavior.

Malicious

The party may arbitrarily deviate from the protocol.

May send incorrect messages, manipulate computation, or reveal information.

Adversary Models in MLaaS

Party	Possible Model	Main Concern
Model Owner	Trusted	Model correctness
Cloud Server	Trusted	Easy security model
Cloud Server	Semi-honest	Privacy / leakage
Cloud Server	Malicious	Privacy + correctness
User	Trusted	Easiest setting
User	Semi-honest	May infer extra information
User	Malicious	May send crafted queries

Key observation:

The security problem becomes harder as we weaken our trust assumptions.

Why Trust the Model Owner?

Suppose the model owner is malicious.

- The owner controls the model itself.
- It can deploy an arbitrary or manipulated model.
- It can claim that the model performs a particular task.
- Cryptography cannot magically make a bad model meaningful.

Practical assumption

We generally assume the **model owner is trusted** with respect to model generation and deployment.
But this does not mean the model must be revealed to everyone.

What About the Cloud Server?

Trusted server

- Simplest setting
- Server can access the model and user data
- Very little cryptographic protection is required

Semi-honest / Malicious server

- Server cannot simply be trusted
- Model and/or user data must be protected
- Need cryptographic techniques
- Encryption, secure computation, ZKP, commitments, etc.

The Post-Quantum Challenge

Suppose we want the MLaaS system to remain secure against quantum adversaries.

- Classical cryptographic solutions may not be sufficient.
- We need **post-quantum secure** primitives.
- Secure computation itself can be expensive.
- Communication can also become a major bottleneck.

Research Challenge

Can we obtain strong security against quantum adversaries while keeping computation and communication practical?

What About the User?

Trusted user

- Simplest setting
- User follows the protocol

Semi-honest user

- Follows the protocol
- May try to infer additional information

Important

The appropriate adversarial assumption depends on the application.

There is no single “secure system” independent of its threat model.

How Do We Design a Secure System?

1 Define the System Model

- Who are the participants?
- What information do they possess?

2 Define the Adversarial Model

- Who can be malicious?
- What can the adversary do?

3 Define Security Goals

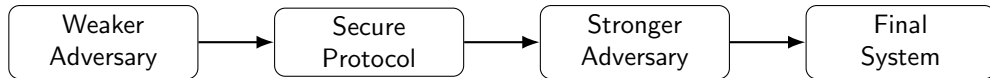
- What exactly must remain private?
- What correctness guarantees are required?

4 Design the Protocol

- Select appropriate cryptographic tools

A Useful Research Strategy

We may not be able to solve the strongest problem immediately.



- Start with a manageable security model.
- Understand the limitations.
- Strengthen the adversarial model.
- Improve the protocol step by step.

This is often how cryptographic research progresses.

What Cryptographic Tools Do We Need?

Once the system and security goals are defined, we ask:

Which cryptographic primitives can achieve these goals?

Possible tools include:

- Encryption
- Bit commitment
- Zero-Knowledge Proofs
- Secure Multi-Party Computation
- Oblivious Transfer
- Homomorphic Encryption
- Secret Sharing

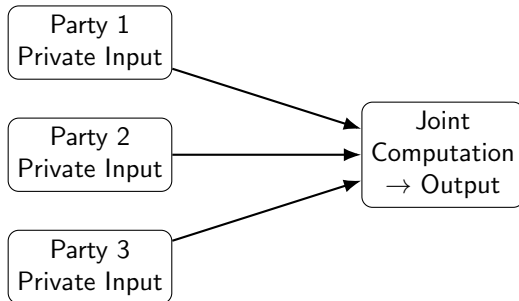
Today: From commitments and ZKP to secure computation

Secure Multi-Party Computation

Idea

Multiple parties jointly compute a function using their private inputs.

- Each party has a private input.
- They interact according to a protocol.
- Everyone obtains the required output.
- No party should learn another party's private input.



MPC: A Simple Example

Suppose three hospitals want to know:

How many patients satisfy a particular condition?

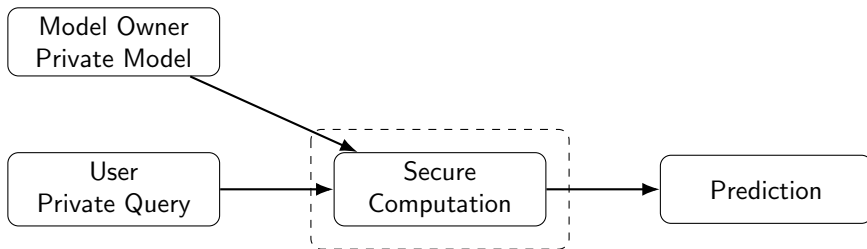
Each hospital has its own private database.

- Hospital A does not reveal its patient records.
- Hospital B does not reveal its patient records.
- Hospital C does not reveal its patient records.
- Yet they jointly obtain the required aggregate result.

Core principle

Compute together without revealing the underlying private inputs.

Where Does MPC Fit in MLaaS?



- User wants to protect the query.
- Owner wants to protect the model.
- Server should learn as little as possible.
- Secure computation may allow inference without exposing raw data.

Take-Home Message

- 1 Cryptography starts with a **security model**, not a primitive.
- 2 Define the participants and their trust assumptions.
- 3 Define exactly what must be protected.
- 4 Then select appropriate cryptographic tools.
- 5 Stronger adversaries generally require stronger and more expensive techniques.
- 6 Bit commitments and ZKP are useful building blocks.
- 7 **MPC** provides a powerful framework for secure joint computation.

Next class: How can we actually build secure computation?

Sources:

- 1 Apostol Vassilev, Alina Oprea, Alie Fordyce, Hyrum Anderson, Xander Davies, and Maia Hamin. *Adversarial Machine Learning: A Taxonomy and Terminology of Attacks and Mitigations*. NIST, 2025. NIST AI 100-2e2025. DOI: 10.6028/NIST.AI.100-2e2025.
- 2 Maria Rigaki and Sebastian Garcia. *A Survey of Privacy Attacks in Machine Learning*. ACM Computing Surveys, 56(4):101–134, 2024. DOI: 10.1145/3624010.

Acknowledgment:

- ChatGPT (OpenAI) was used to identify the content for the slides, which was subsequently verified by me.



Dr. Laltu Sardar, Assistant Professor, IISER Thiruvananthapuram
laltu.sardar@iisertvm.ac.in, laltu.sardar.crypto@gmail.com
Course webpage: https://laltu-sardar.github.io/courses/26_crypto.html.